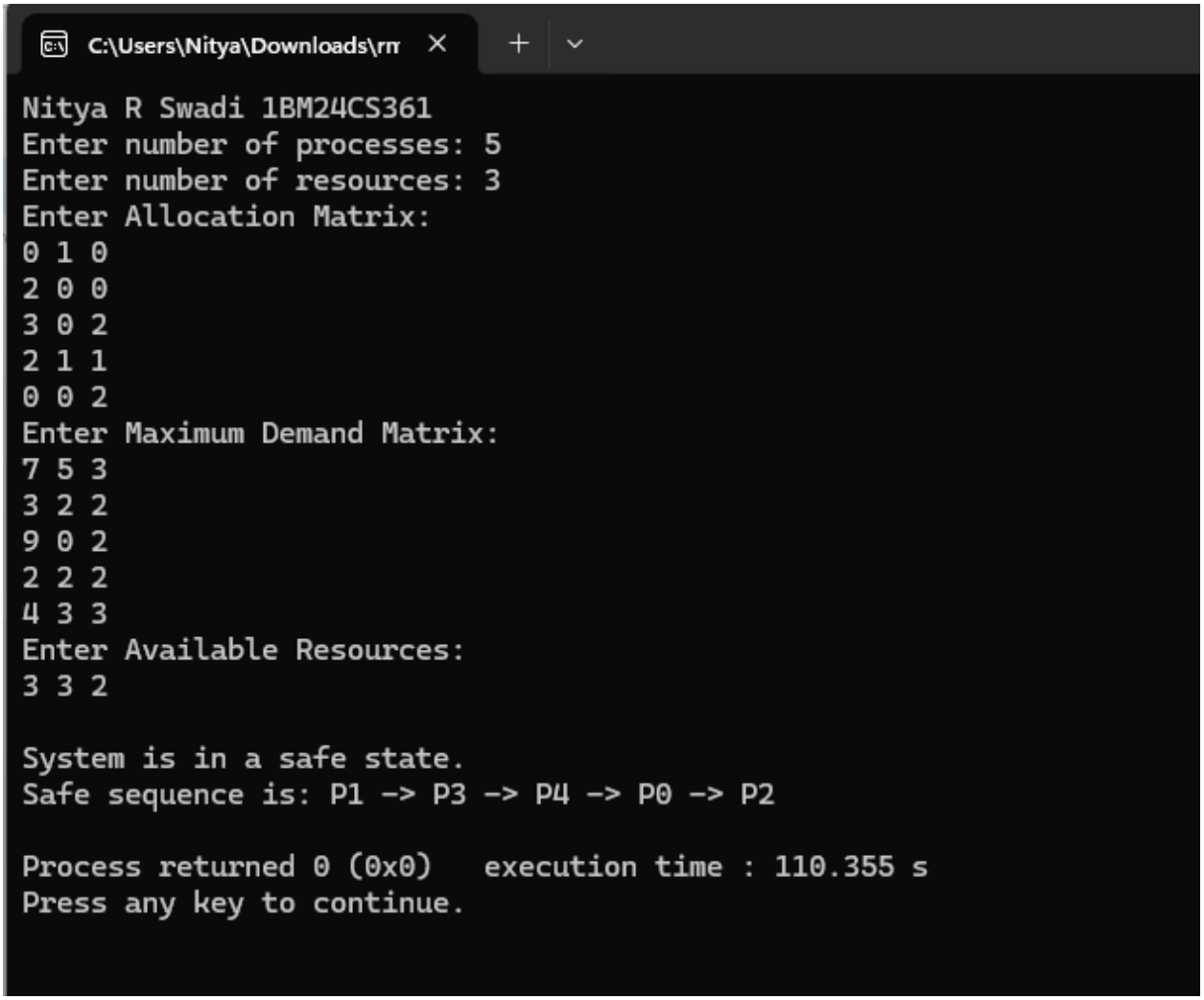
Safety Algorithm

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    // 1. Input configuration
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m];
    int max[n][m];
    int avail[m];
    int need[n][m];

    // 2. Input matrices
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Maximum Demand Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
            // Calculate Need matrix: Need = Max - Allocation
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
```

```
    printf("Enter Available Resources:\n");
    for (int j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }

    // 3. Initialize Safety Algorithm tracking structures
    int work[m];
    for (int j = 0; j < m; j++) {
        work[j] = avail[j];
    }

    bool finish[n];
    for (int i = 0; i < n; i++) {
        finish[i] = false;
    }

    int safe_sequence[n];
    int count = 0;

    // 4. Safety Algorithm Loop
    for (int k = 0; k < n; k++) { // Maximum possible iterations is equal to number of
processes
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                bool can_allocate = true;

                // Check if Need <= Work for all resources
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > work[j]) {
                        can_allocate = false;
                        break;
                    }
                }

                // If process demands can be met
                if (can_allocate) {
                    // Reclaim resources
```

```
                for (int j = 0; j < m; j++) {
                    work[j] += alloc[i][j];
                }
                finish[i] = true;
                safe_sequence[count++] = i;
            }
        }
    }
}

// 5. Check final state and print matching output
printf("\n");
if (count == n) {
    printf("System is in a safe state.\n");
    printf("Safe sequence is: ");
    for (int i = 0; i < n; i++) {
        printf("P%d", safe_sequence[i]);
        if (i < n - 1) {
            printf(" -> ");
        }
    }
    printf("\n");
} else {
    printf("System is not in a safe state (Deadlock imminent).\n");
}

return 0;
}
```

OUTPUT:


C:\Users\Nitya\Downloads\rr
Nitya R Swadi 1BM24CS361
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0) execution time : 110.355 s
Press any key to continue.